

Clase 206 — Análisis de artefactos de Linux

Parte: 9 — Forense digital y respuesta a incidentes · Fuente: NIST SP 800-86 y documentación de systemd/journald 🕒 Duración estimada: 120 min · Nivel: Avanzado

Objetivo

Aprender dónde vive la evidencia en un sistema Linux comprometido: logs de sistema (syslog/journald), historial de shell, cron/systemd timers, cuentas y autenticación, y persistencia común de atacantes. Al terminar podrás reconstruir la actividad de un usuario y de un intruso en Linux.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Analizar** logs de autenticación y sistema en `/var/log` y `journald`.
- 2. **Reconstruir** actividad de usuario con historiales de shell y timestamps.
- 3. **Detectar** mecanismos de persistencia (cron, systemd, rc, bashrc).
- 4. **Interpretar** cuentas, sudoers y claves SSH sospechosas.
- 5. **Correlacionar** artefactos para trazar una intrusión.

Temas

#	Tema	Por qué importa
1	<code>/var/log</code> y syslog	Registro central del sistema
2	journald (systemd)	Logs binarios modernos
3	auth.log / secure y wtmp/btmp	Autenticación y sesiones
4	Historial de shell	Comandos ejecutados
5	cron y systemd timers	Persistencia programada
6	Cuentas, sudoers, SSH keys	Acceso y escalado
7	Timestamps y <code>stat</code>	Orden de eventos
8	Persistencia típica de atacantes	Qué buscar

Definiciones y características

- **syslog**: sistema clásico de logs de texto en `/var/log`. Característica: legible con herramientas estándar.
- **journald**: registro binario de systemd, se lee con `journalctl`. Característica: incluye metadatos ricos y es más difícil de manipular en texto plano.

- **wtmp/btmp/lastlog**: registros binarios de logins exitosos, fallidos y último acceso. Característica: se leen con `last`, `lastb`, `lastlog`.
- **Historial de shell**: `~/.bash_history`, `~/.zsh_history`. Característica: puede tener timestamps si `HISTTIMEFORMAT` está activo.
- **cron / systemd timer**: tareas programadas. Característica: vías comunes de persistencia.
- **authorized_keys**: claves SSH que permiten acceso sin contraseña. Característica: una clave añadida es señal de backdoor.
- **stat**: muestra timestamps atime/mtime/ctime de un archivo. Característica: base del timeline en Linux.

Herramientas y preparación

- **Nativas**: `journalctl`, `last`, `lastb`, `grep`, `stat`, `ausearch` (auditd).
- **Análisis**: The Sleuth Kit para montar la imagen, `log2timeline` / `plaso` para timeline.
- **Entorno**: usa una imagen de una VM Linux propia. Monta en solo lectura:

```
bash mount -o ro,noexec,nodev imagen.dd /mnt/evidencia
```

Laboratorio guiado

Trabaja sobre una imagen de una VM Linux propia montada en solo lectura.

1. Revisa autenticación (SSH, sudo):

```
bash grep -Ei "accepted|failed|sudo" /mnt/evidencia/var/log/auth.log
```

2. Lee logins con los registros binarios:

```
bash last -f /mnt/evidencia/var/log/wtmp lastb -f /mnt/evidencia/var/log/btmp
```

3. Consulta journald offline:

```
bash journalctl --directory=/mnt/evidencia/var/log/journal --no-pager
```

4. Extrae el historial de shell de cada usuario:

```
bash cat /mnt/evidencia/home/*/.bash_history cat /mnt/evidencia/root/.bash_history
```

5. Busca persistencia programada:

```
bash ls -la /mnt/evidencia/etc/cron* cat /mnt/evidencia/var/spool/cron/crontabs/* ls -la /mnt/evidencia/etc/systemd/system/
```

6. Revisa cuentas y accesos sospechosos:

```
bash cat /mnt/evidencia/etc/passwd | awk -F: '$3 ≥ 1000' cat /mnt/evidencia/etc/sudoers cat /mnt/evidencia/home/*/.ssh/authorized_keys
```

7. Revisa persistencia en perfiles de shell:

```
bash grep -R . /mnt/evidencia/home/*/.bashrc /mnt/evidencia/etc/rc.local 2>/dev/null
```

8. Ordena hallazgos por tiempo con `stat` sobre archivos sospechosos.

Ejercicios

1. Extrae de `auth.log` todos los logins SSH exitosos y su IP de origen.
2. Detecta un ataque de fuerza bruta contando fallos en `btmpt`.
3. Encuentra una tarea cron maliciosa que tú mismo plantaste.
4. Identifica una cuenta con UID 0 distinta de `root`.
5. Explica la diferencia entre `atime`, `mtime` y `ctime`.
6. Reconstruye la secuencia de comandos de un usuario desde su historial.

Reto verificable

En una VM Linux propia, simula una intrusión (crea un usuario extra con UID 0, añade una clave SSH y un cron de persistencia) y luego, desde la imagen, detecta y documenta los tres mecanismos.

Criterio de aceptación: tu informe identifica el usuario malicioso, la clave SSH añadida y la tarea de persistencia, cada uno con la ruta del artefacto, el timestamp relevante y el comando que lo reveló.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>.bash_history</code> vacío	Atacante lo borró o usó <code>unset HISTFILE</code> . Busca en <code>journal</code> o <code>timeline</code> .
<code>journalctl</code> no lee la carpeta	Falta <code>--directory</code> correcto o <code>journal</code> corrupto. Verifica la ruta.
Timestamps alterados	El atacante usó <code>touch</code> . Contrasta con <code>ctime</code> y con logs.
Montaste con <code>exec</code>	Riesgo de ejecutar malware. Usa siempre <code>ro,noexec,nodev</code> .
No ves los logins	Logs rotados/comprimidos. Descomprime <code>auth.log.*.gz</code> .

Preguntas frecuentes

 **¿journald reemplaza a syslog?** Coexisten. `journald` es el estándar en `systemd`; muchos sistemas también reenvían a `syslog` en texto.

 **¿El historial de shell tiene fechas?** Solo si `HISTTIMEFORMAT` estaba configurado. Si no, tienes el orden pero no la hora exacta.

 **¿Cómo detecto persistencia?** Revisa `cron`, `systemd timers/services`, `rc.local`, perfiles de shell, y `authorized_keys`. Compara contra un baseline limpio.

 **¿ctime se puede falsificar?** `mtime` y `atime` sí con `touch`; `ctime` es más difícil (requiere manipular el reloj o el FS), por eso es más confiable.

Referencias

- NIST SP 800-86: <https://csrc.nist.gov/publications/detail/sp/800-86/final>
- `systemd journald` docs: <https://www.freedesktop.org/software/systemd/man/journalctl.html>

- The Sleuth Kit: <https://www.sleuthkit.org/>
- SANS — Linux forensics resources: <https://www.sans.org/blog/>

Siguiente clase

Clase 207 - Forense de memoria RAM con Volatility